Vergleich von Rekursion und Iteration

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Terminierung
- Muster für Abbruchbedingungen
- Muster für Fortschritt

Was wird in diesem Foliensatz besprochen?

- Vergleich von Iteration und Rekursion
- Stack-Speicher

Was lernen Sie kennen?

- Unterschiede bei Iteration und Rekursion
- Stack-Speicher in Java
- Rekursion und Stack-Speicher

Rekursion und Iteration

Rekursion und Iteration sind gleich mächtig

Rekursion lässt sich in Iteration umformen und umgekehrt

- Das sagt noch nichts über die Lesbarkeit, den Umfang und die Effizienz des Ergebnisses aus

Umwandlung von Rekursion zu Iteration

Einfach bei Endrekursion und linearer Preorder-Rekursion

- Manche Compiler erkennen Endrekursion und formen automatisch um (nicht bei Oracle-Compiler für Java)

Meist umfangreicher bei linearer Postorder-Rekursion und verzweigter Rekursion

Umfangreicher bei verschachtelter Rekursion

Rekursion und Iteration im Vergleich

Innerhalb eines Methodenrumpfs können mehrere rekursive Aufrufe erfolgen

- Es ist aufwendig (aber möglich), solche rekursiven Methoden in Schleifen umzuformen

Bestimmte Aufgaben lassen sich mit Rekursion kürzer formulieren

Daten weiterreichen

Bei der Rekursion können Daten weitergereicht werden

- Durch Parameter
- Durch Rückgabewerte, die wiederum in Berechnungen einfließen können

In einer Schleife können Daten dagegen nur von der vorherigen zur nächsten Iteration fließen

- Zur Kompensation sind häufig zusätzliche Variablen für Zwischenergebnisse notwendig
- Komplexere Struktur durch zusätzliche Variablen

Umgang mit Speicher

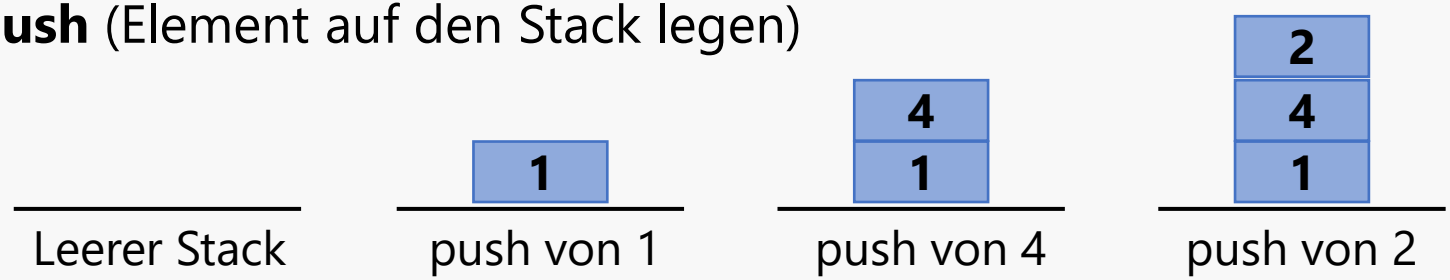
Schleifen gehen mit dem Speicher (zumindest in Java) weniger verschwenderisch um

Rekursion braucht mehr Speicher!

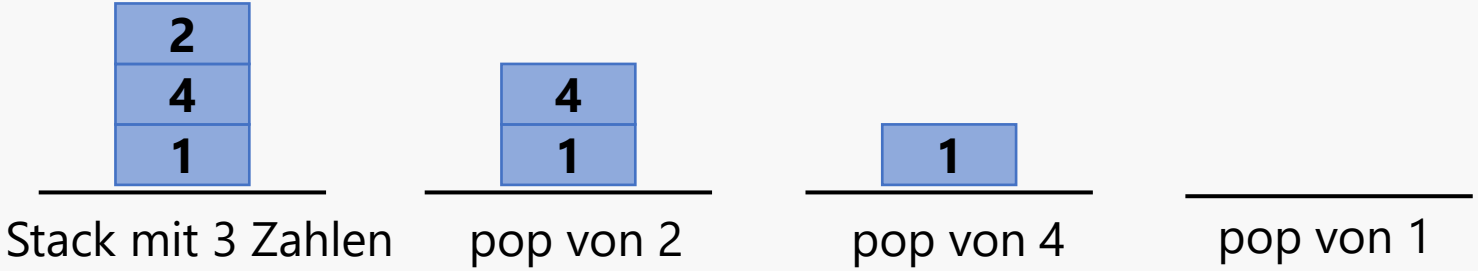
Stack-Speicher

Datenstruktur Stack

- Operationen (z. B. für einen Stack von Zahlen jeweils nach der Operation)
  - **push** (Element auf den Stack legen)



  - **pop** (oberstes Element vom Stack nehmen)



JVM-Stack und Methoden-Frames (1)

JVM-Stack

- Beim Ablauf eines Programms
- Hat eine begrenzte Größe

Stack-Frame wird beim Aufruf einer Methode erzeugt

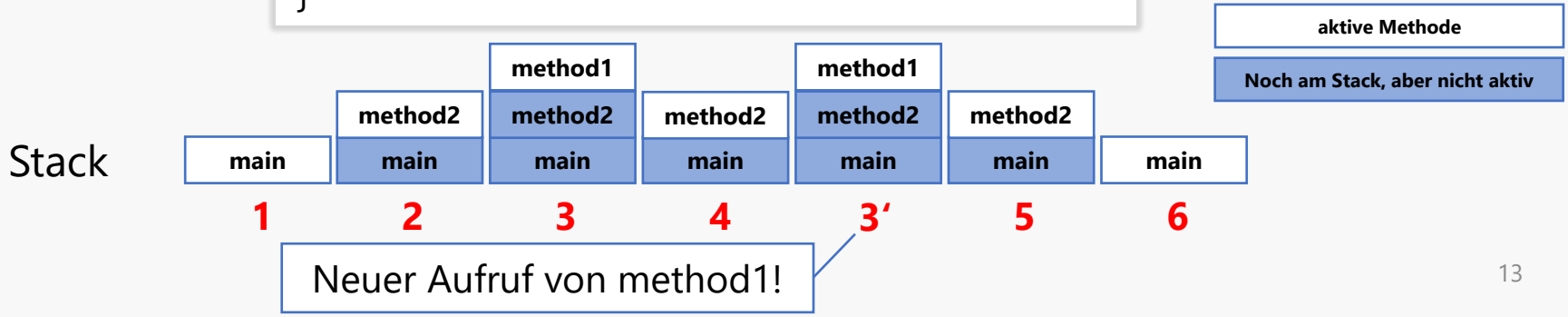
- Speichert Parameter, lokale Variablen, Zwischenresultate und weitere Daten einer Methode
- Wird oben auf den Stack gelegt und wird der aktuelle (aktive) Frame

JVM-Stack und Methoden-Frames (2)

Bei Beendigung einer Methode	
Java Virtual Machine nimmt den Frame vom Stack und verwirft ihn (lokale Daten gehen verloren)	Der Frame der vorhergehenden Methode (aufrufenden Methode) wird der aktuelle (aktive) Frame

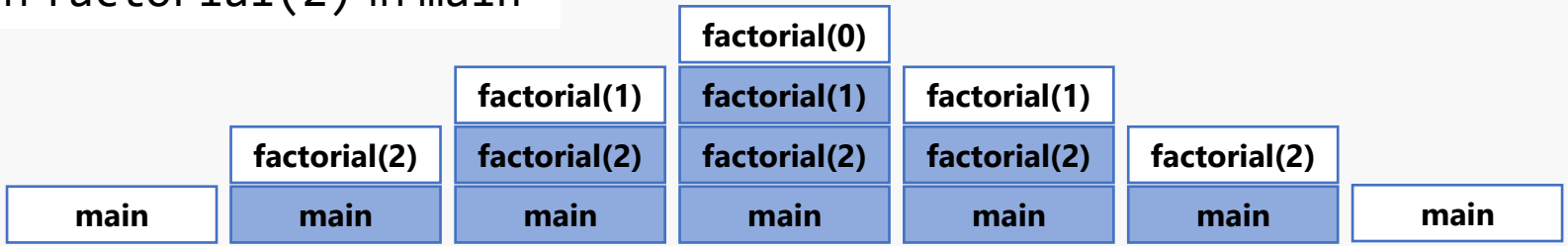
Stack bei Methodenaufrufen

```
public class Tester {
    public static void main(String[] args) {
        1 … method2(); … 6
    }

    private static void method2() {
        2 … method1(); … 4 … method1(); … 5
    }

    private static void method1() {
        … 3 …
    }
}
```

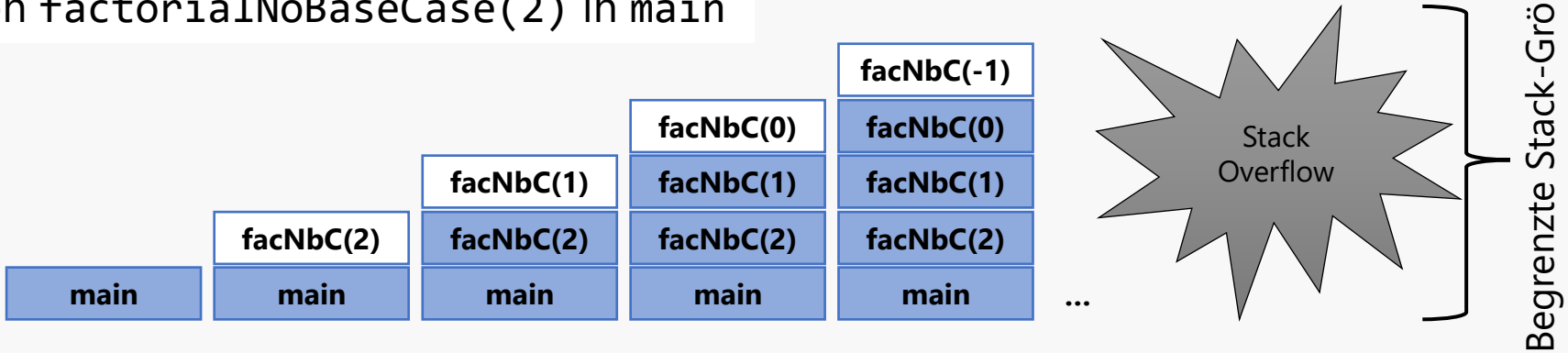


Stack und Rekursion (1)

Aufruf von `factorial(2)` in `main`



Aufruf von `factorialNoBaseCase(2)` in `main`



Stack und Rekursion (2)

```
private static int sumRecursive(int num) {
    if (num <= 1) {
        return 1;
    } else {
        return sumRecursive(num - 1) + num;
    }
}
```

Terminierung
• Fundiert • Fortschritt korrekt

Praktisches Problem

- Zu viele rekursive Aufrufe bei großen Werten von *n* führen wiederum zu einem Stack-Overflow (Iteration dann vorteilhaft)

Abbruch

Falsche oder fehlende Abbruchbedingung

- Weder rekursive Methoden noch Schleifen terminieren auf normale Weise

Rekursive Methoden brechen (zumindest in Java) irgendwann ab

- Stack-Overflow

Schleifen können ohne Fehlermeldung endlos laufen

- Aufgrund der Fehlermeldungen sind solche Fehler bei Rekursion etwas leichter zu identifizieren als bei Schleifen

Methodengröße

Rekursion führt zu vielen kleinen Methoden, Schleifen zu eher größeren

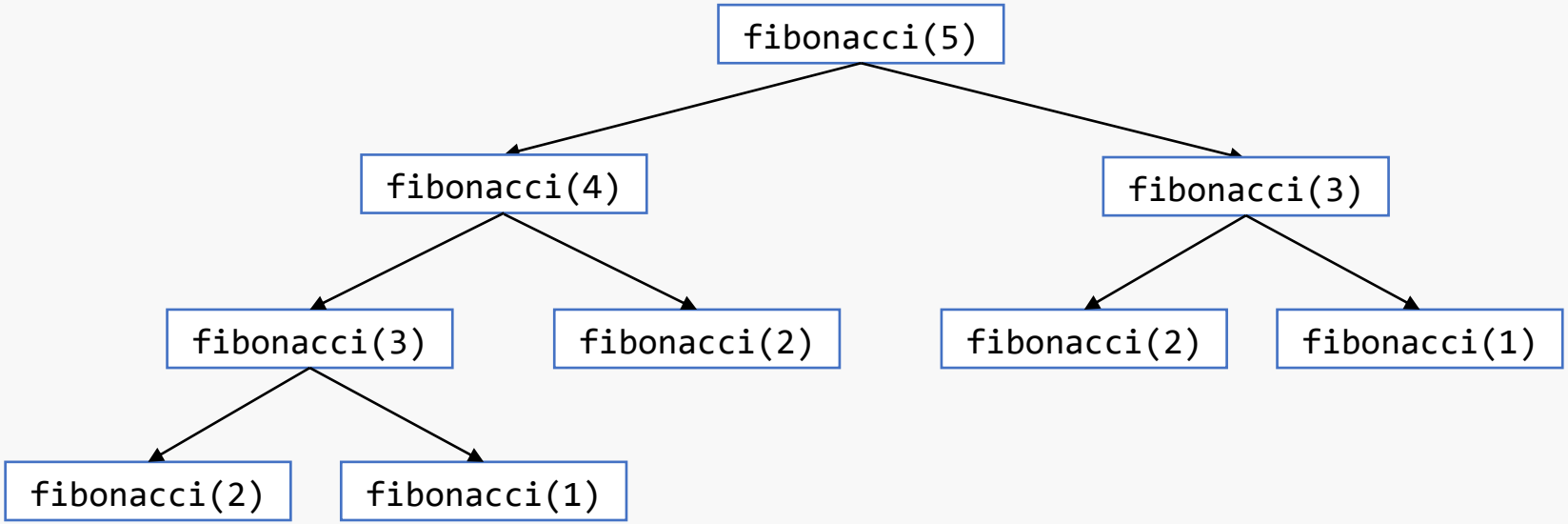
Bei Schleifen ist die Größe der Methoden besser steuerbar

Effizienz

Rekursion kann **sehr ineffizient** sein - Beispiel Fibonacci-Zahlen

```
private static long fibonacci(int n) {
    if (n <= 2) {
        return 1;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Aufrufbaum (Wiederholung)



Wie sieht das für fibonacci(50) aus?

- Wie viele Aufrufe müssen dort durchgeführt werden?

Test (Zeit messen, Anzahl der Aufrufe zählen)

```
public class FibonacciTester {
    private static long counter;
    // n >= 1
    private static long fibonacci(int n) {
        counter++;
        return n <= 2 ? 1 : fibonacci(n - 1) + fibonacci(n - 2);
    }
    public static void main(String[] args) {
        long start, end;
        long result;
        for (int i = 1; i <= 50; i++) {
            counter = 0;
            start = System.nanoTime();
            result = fibonacci(i);
            end = System.nanoTime();
            System.out.print("fibonacci(" + i + ") = " + result + " -> " + (end - start) / 1000000000.0
            + " seconds for " + counter + " method calls\n");
        }
    }
}
```

Ausgabe des Testers (Beispiel)

```
...
fibonacci(40) = 102334155 -> 0.258819501 seconds for 204668309 method calls
fibonacci(41) = 165580141 -> 0.4036408 seconds for 331160281 method calls
fibonacci(42) = 267914296 -> 0.6514712 seconds for 535828591 method calls
fibonacci(43) = 433494437 -> 1.0455163 seconds for 866988873 method calls
fibonacci(44) = 701408733 -> 1.611623599 seconds for 1402817465 method calls
fibonacci(45) = 1134903170 -> 2.611942001 seconds for 2269806339 method calls
fibonacci(46) = 1836311903 -> 4.2654961 seconds for 3672623805 method calls
fibonacci(47) = 2971215073 -> 6.8933712 seconds for 5942430145 method calls
fibonacci(48) = 4807526976 -> 11.134899901 seconds for 9615053951 method calls
fibonacci(49) = 7778742049 -> 18.0095478 seconds for 15557484097 method calls
fibonacci(50) = 12586269025 -> 29.3449949 seconds for 25172538049 method calls
```

Testergebnisse

Anzahl der Methodenaufrufe ist fix für ein bestimmtes n

- Nimmt stark zu, führt aber zu keinem Abbruch!
- Aufruftiefe ist nicht sehr groß

Zeitangaben sind keine fixen Werte

- Auf einem konkreten Rechner (mit Intel Core i7 9750H) gemessen
- Können leicht schwanken
- Zeit für fibonacci(n) ≈ Zeit für fibonacci(n-1) + Zeit für fibonacci(n-2)

fibonacci(100)?

Wie lange dauert fibonacci(100)?

ca. 26 000 Jahre (eine sehr grobe Abschätzung!)

Wie kann das rekursiv und effizient implementiert werden?

Fibonacci-Zahlen endrekursiv

```
private static long fibonacci2(int n, long current, long next) {
    if (n <= 1) {
        return current;
    } else {
        return fibonacci2(n - 1, next, current + next);
    }
}

private static long fibonacci2(int n) {
    return fibonacci2(n, 1, 1);
}
```

Beispiel für Ablauf bei fibonacci2(10)

n	current	next
10	1	1
9	1	2
8	2	3
7	3	5
6	5	8
5	8	13
4	13	21
3	21	34
2	34	55
1	**55**	89